UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

SISTEMA DE UNIVERSIDAD ABIERTA

Alumno: Emanuel Marzano Villalobos

Programación De Dispositivos Móviles

Cristian Cardoso Arellano

Grupo 9697

m4_01

15/10/2026

Enlace a repositorio de la actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M4_01

1. Introducción

Cuando usamos el celular, abrimos una aplicación, la minimizamos para contestar un mensaje, regresamos a ella o la giramos para ver un video en pantalla completa. Desde fuera parece que la aplicación simplemente “está ahí”, pero en realidad Android está tomando decisiones todo el tiempo: cuándo mostrarla, cuándo pausarla, cuándo mandarla a segundo plano y cuándo eliminarla de la memoria para ahorrar recursos.

Cada pantalla de una aplicación Android se llama Activity (actividad), y el conjunto de etapas por las que pasa desde que se crea hasta que se destruye se conoce como su ciclo de vida. En cada cambio de etapa, Android ejecuta automáticamente un método especial de la actividad. Si el programador escribe código dentro de esos métodos, puede decidir qué hacer en cada momento.

En esta actividad se crea una aplicación sencilla en Java que imprime un mensaje personalizado en Logcat dentro de cada uno de los siete métodos del ciclo de vida. Después, se realizan acciones con la aplicación en el emulador (abrirla, minimizarla, regresar a ella y girar la pantalla) para observar en qué orden y en qué momento Android llama a cada método.

2. Objetivo

Comprender el ciclo de vida de una actividad en Android mediante la impresión de un mensaje personalizado en cada uno de sus métodos (onCreate, onStart, onResume, onPause, onStop, onRestart y onDestroy), observando en Logcat el orden en que se ejecutan ante distintas acciones del usuario.

3. ¿Qué es el ciclo de vida de una Activity?

3.1. Una comparación para entenderlo

Antes de ver el código, ayuda pensar en la actividad como si fuera una tienda en un centro comercial:

- **onCreate()** es cuando se construye la tienda por primera vez: se acomodan los estantes, los productos y los letreros. Esto solo se hace una vez.
- **onStart()** es cuando se sube la cortina: la gente ya puede ver la tienda desde el pasillo, pero todavía no puede entrar.
- **onResume()** es cuando se abre la puerta: ahora los clientes pueden entrar y comprar. Es el momento en que la aplicación está “en uso”.
- **onPause()** es cuando alguien se para frente a la puerta y estorba: la tienda sigue ahí, pero ya nadie puede entrar en ese momento.
- **onStop()** es cuando se baja la cortina: la tienda ya no se ve, pero todo sigue acomodado adentro.

- **onRestart()** es cuando se vuelve a abrir la tienda después de haber bajado la cortina. Como todo seguía acomodado, no hay que construirla de nuevo.
- **onDestroy()** es cuando la tienda se desmonta por completo. Si se quisiera abrir otra vez, habría que construirla desde cero.

La idea más importante es que el programador no llama a estos métodos. Es Android quien los ejecuta automáticamente según lo que pasa con la aplicación. El programador solo decide qué código poner dentro de cada uno.

3.2. Diagrama del ciclo de vida

El siguiente diagrama resume el recorrido. Las flechas grises muestran el camino normal de arriba hacia abajo; las verdes, los regresos posibles; y la naranja punteada, lo que ocurre al girar la pantalla.

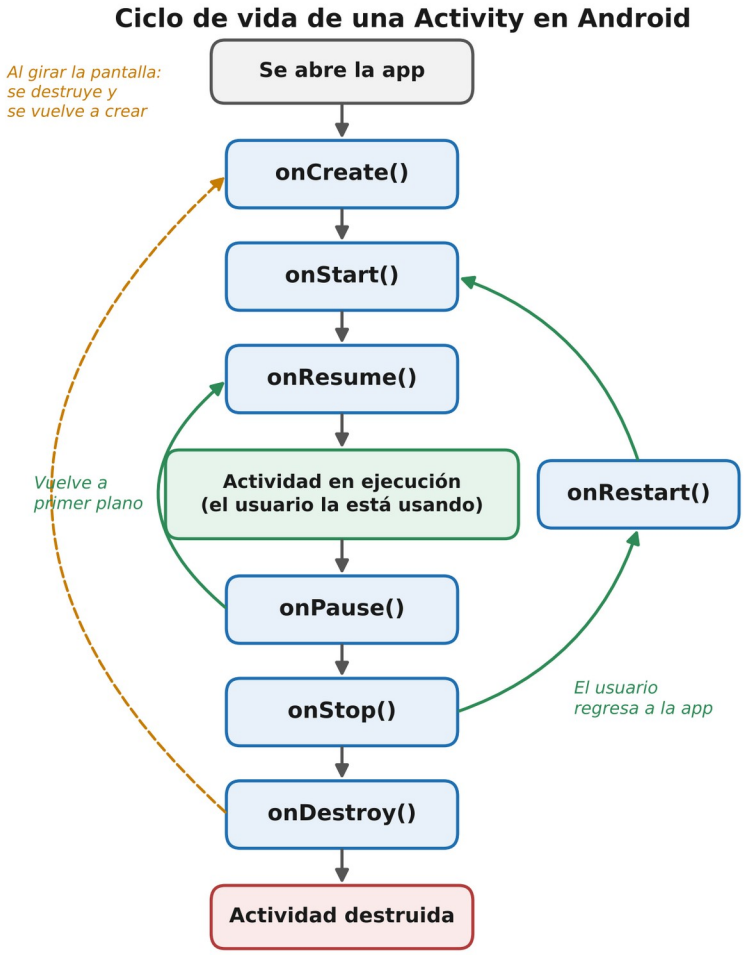


Figura 1. Ciclo de vida de una Activity (elaboración propia basada en la documentación de Android).

3.3. Los siete métodos

Método	¿Cuándo lo llama Android?	¿En qué estado queda?	Ejemplo de uso en una app real
`onCreate()`	Una sola vez, cuando la actividad se crea.	Creada, pero todavía no se ve.	Cargar el diseño (setContentView), preparar botones y variables.
`onStart()`	Justo antes de que la actividad aparezca en pantalla.	Visible, pero aún no recibe toques.	Empezar a mostrar información actualizada.
`onResume()`	Cuando la actividad pasa al frente y el usuario puede usarla.	En ejecución (primer plano).	Reanudar un video, la cámara o una animación.
`onPause()`	Cuando la actividad empieza a perder el foco.	Pausada: puede verse parcialmente, pero no es la principal.	Pausar un video o una canción.
`onStop()`	Cuando la actividad deja de verse por completo.	Detenida: sigue en memoria, en segundo plano.	Guardar cambios, liberar recursos pesados.
`onRestart()`	Cuando una actividad detenida vuelve a abrirse.	Regresa, y enseguida se llama onStart().	Volver a cargar datos que pudieron cambiar.
`onDestroy()`	Antes de que la actividad se elimine de la memoria.	Destruida.	Limpieza final de recursos.

Para recordar: los métodos van en parejas

onCreate ↔ **onDestroy**: la actividad nace y muere.

onStart ↔ **onStop**: la actividad aparece y desaparece de la pantalla.

onResume ↔ **onPause**: la actividad gana y pierde el foco (si el usuario puede usarla o no).

onRestart es el único sin pareja: solo ocurre cuando la actividad regresa después de un onStop.

4. Desarrollo

4.1. Creación del proyecto

Se creó el proyecto M4_01 con la plantilla Empty Views Activity y el lenguaje Java. Android Studio generó la actividad principal MainActivity.java, que ya incluye el método onCreate().

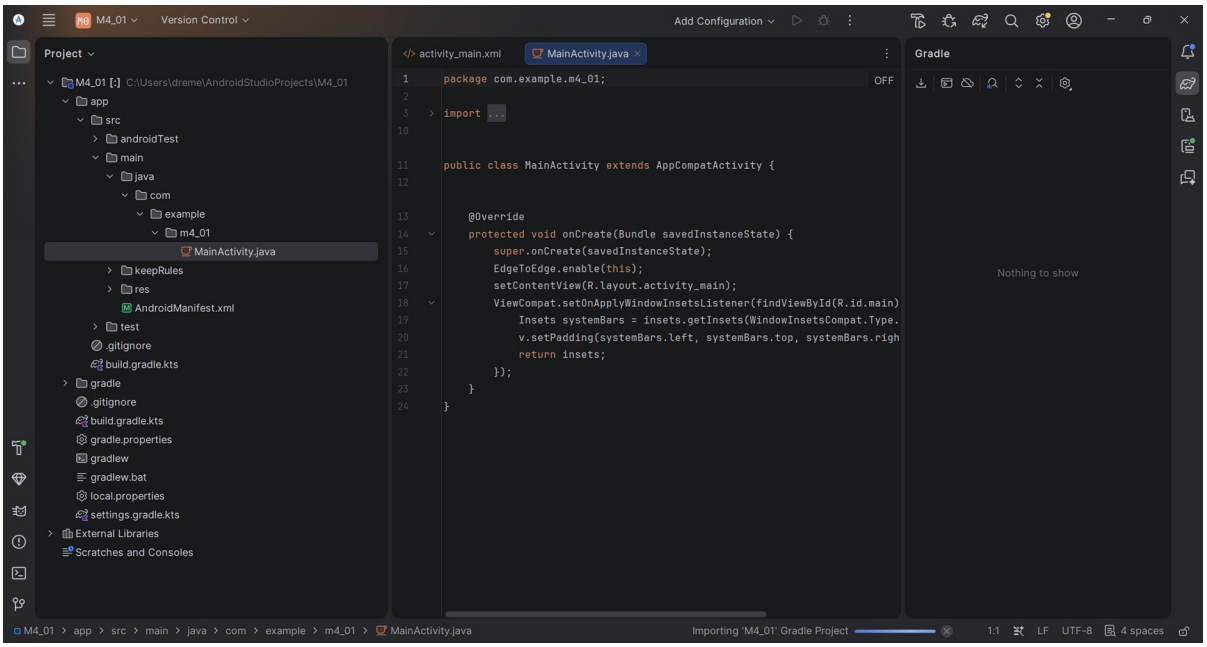


Figura 2. Proyecto M4_01 recién creado.

La plantilla solo trae onCreate(), porque es el único método obligatorio para mostrar la pantalla. Los demás existen “escondidos” en la clase AppCompatActivity, de la cual hereda MainActivity, y se pueden reescribir cuando se necesitan.

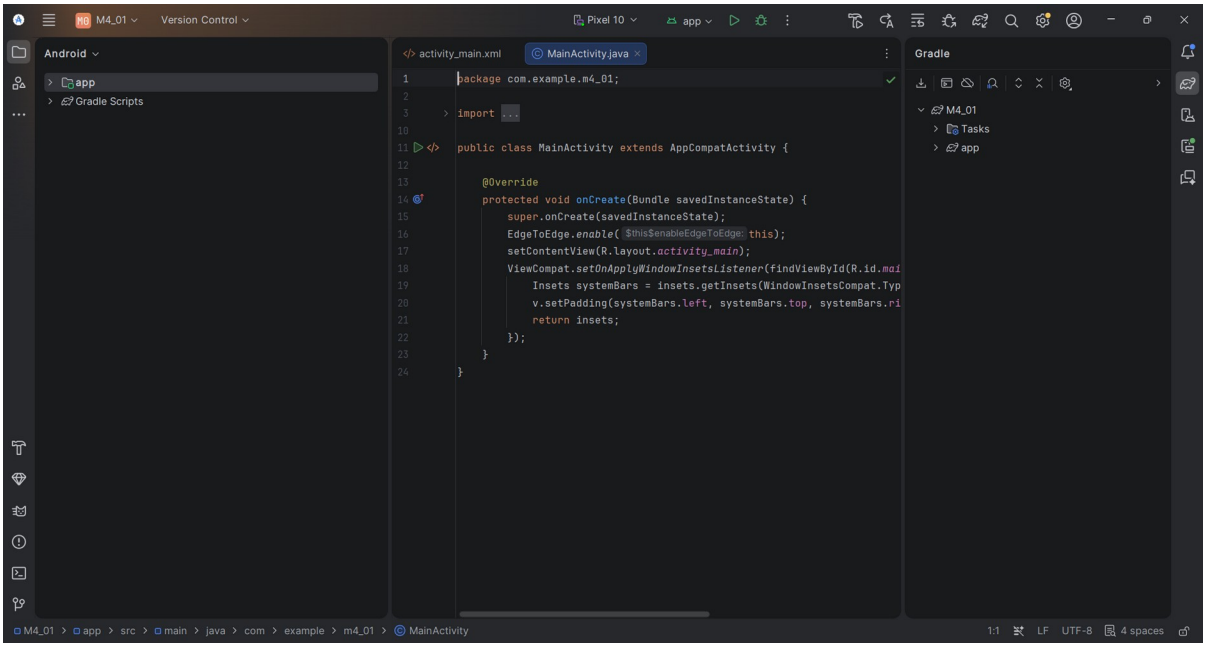


Figura 3. MainActivity.java como lo genera la plantilla.

4.2. Código agregado

La etiqueta TAG

```
private static final String TAG = "M4_01";
```

- **`private static final String TAG`**: crea una constante de texto con el valor "M4_01". Se usa como etiqueta en todos los mensajes para poder filtrarlos en Logcat. Al ser una constante, si se quisiera cambiar la etiqueta bastaría con modificar esta línea.

El mensaje en onCreate()

Al final del método onCreate() que ya existía se agregó una línea:

```
Log.d(TAG, "onCreate: la actividad se está creando");
```

- **`Log.d`**: imprime un mensaje de depuración (debug) en Logcat.
- **`TAG`**: es la etiqueta definida arriba.
- **`"onCreate: …"`**: es el mensaje personalizado. Empieza con el nombre del método para identificar fácilmente cuál se ejecutó.

Los otros seis métodos

Después de onCreate(), y dentro de la clase, se escribieron los otros seis métodos. Todos siguen la misma estructura; por ejemplo, onStart():

```
@Override
protected void onStart() {
    super.onStart();
    Log.d(TAG, "onStart: la actividad está por hacerse visible");
}
```

- **`@Override`**: indica que se está reescribiendo un método que ya existe en la clase padre (AppCompatActivity). Es como decirle a Android: “cuando llegue este momento, además de lo que tú haces, ejecuta también lo mío”.
- **`protected void onStart()`**: protected significa que el método se puede usar dentro de la clase y sus clases hijas; void, que no devuelve ningún valor. El nombre debe escribirse exactamente igual, porque Android lo busca por ese nombre.
- **`super.onStart();`**: ejecuta primero el código original de Android para ese momento. Es obligatorio: si se omite, la aplicación se detiene con un error, porque Android necesita hacer su propio trabajo en cada etapa.
- **`Log.d(TAG, …)`**: imprime el mensaje personalizado de ese método.

El código completo de los seis métodos agregados es el siguiente:

```
@Override
protected void onStart() {
    super.onStart();
    Log.d(TAG, "onStart: la actividad está por hacerse visible");
}

@Override
protected void onResume() {
    super.onResume();
    Log.d(TAG, "onResume: la actividad está en primer plano");
}

@Override
protected void onPause() {
    super.onPause();
    Log.d(TAG, "onPause: la actividad pierde el foco");
}

@Override
protected void onStop() {
    super.onStop();
    Log.d(TAG, "onStop: la actividad ya no es visible");
}

@Override
protected void onRestart() {
    super.onRestart();
    Log.d(TAG, "onRestart: la actividad vuelve después de haberse
detenido");
}

@Override
protected void onDestroy() {
    super.onDestroy();
    Log.d(TAG, "onDestroy: la actividad se está destruyendo");
}
```

4.3. Error durante el desarrollo y su corrección

Al agregar los seis métodos, Android Studio marcó 25 errores. La causa fue la posición de una llave: la llave que cierra la clase (la que está pegada al margen izquierdo) quedó justo después de onCreate(), por lo que los seis métodos nuevos quedaron fuera de la clase. En Java, todo método debe estar dentro de una clase, por eso Android Studio no los reconocía.

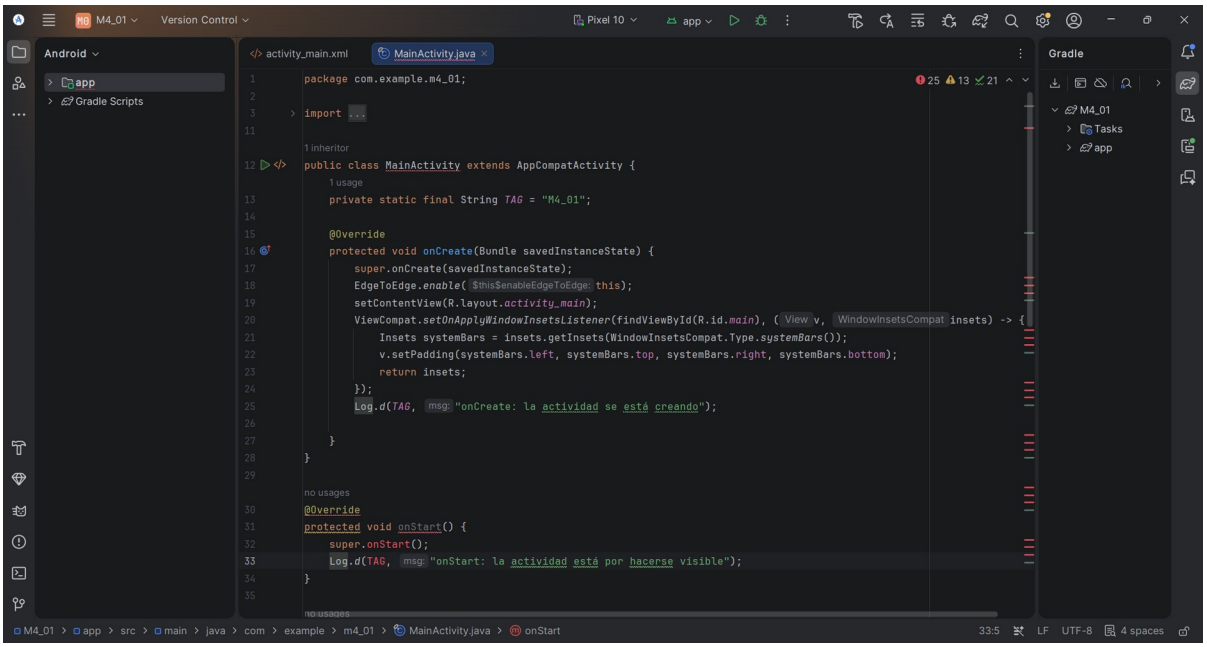


Figura 4. Los métodos quedaron fuera de la clase y se marcaron 25 errores.

Para corregirlo, se borró esa llave y se colocó al final del archivo, después de onDestroy(). Luego se usó el atajo Ctrl + Alt + L (Reformat Code), que acomoda automáticamente la sangría y permite ver con claridad que todos los métodos quedaron alineados dentro de la clase. Los errores desaparecieron.

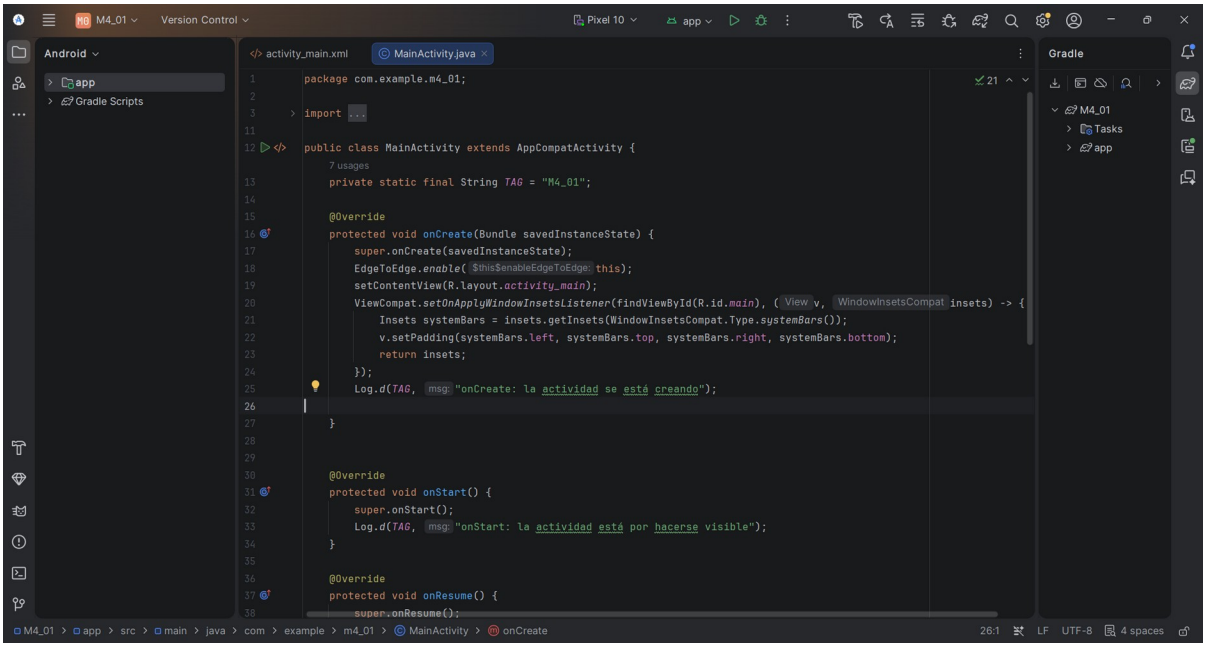


Figura 5. Código corregido: TAG, onCreate() y onStart() dentro de la clase.

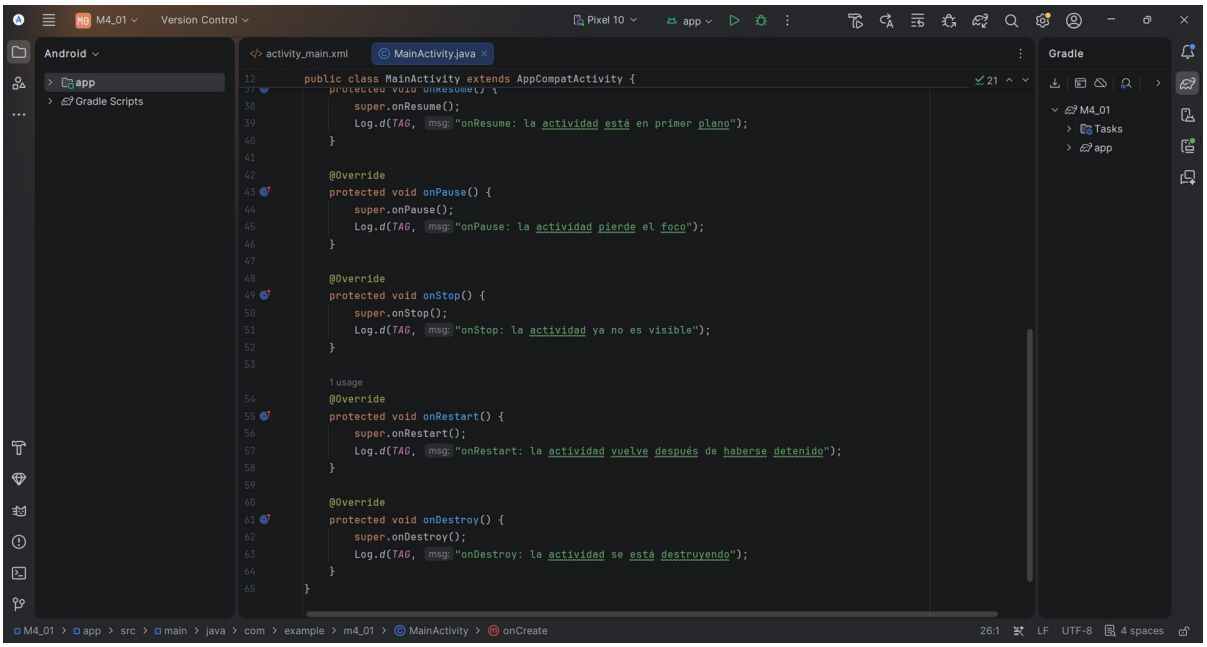


Figura 6. Código corregido: el resto de los métodos, con la llave de la clase al final (línea 65).

4.4. Pruebas: observando el ciclo de vida

La aplicación se ejecutó en el emulador Pixel 10 (Android 17, API 37). En Logcat se aplicó el filtro tag:M4_01 para ver solo los mensajes de esta práctica. Después se realizaron cuatro acciones, una a la vez, observando qué métodos aparecían en cada una. Los botones del emulador se encuentran en la barra superior del panel Running Devices.

Prueba 1: abrir la aplicación

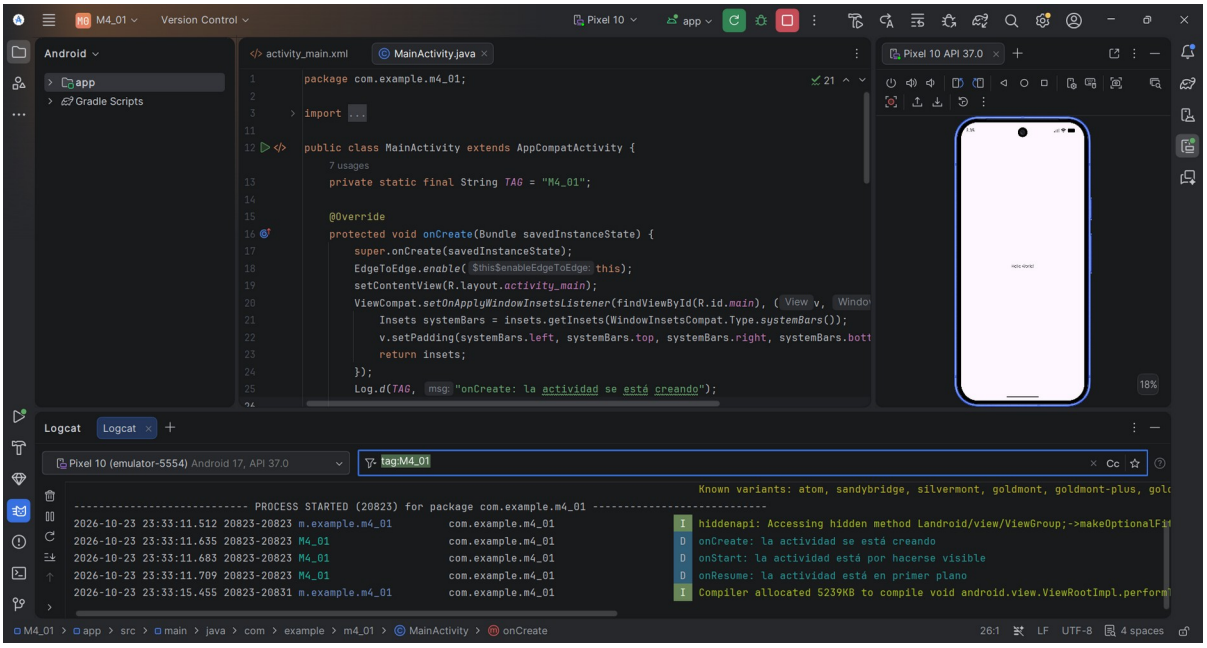


Figura 7. Al abrir la aplicación: onCreate → onStart → onResume.

Descripción: Al iniciar la aplicación aparecieron tres mensajes, siempre en el mismo orden: primero la actividad se crea (onCreate), después se hace visible (onStart) y, por último, pasa al frente para que el usuario pueda usarla (onResume). Aunque ocurren en milésimas de segundo, las horas de Logcat muestran que son tres pasos distintos.

Prueba 2: minimizar la aplicación (botón Inicio ○)

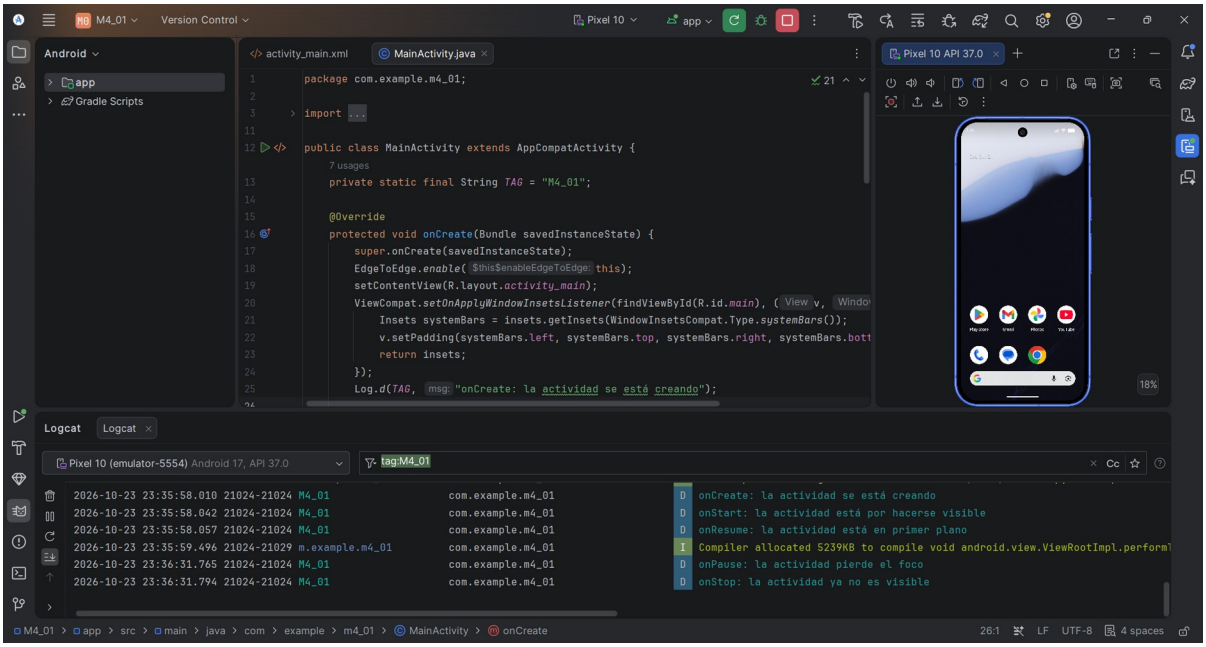


Figura 8. Al presionar Inicio: onPause → onStop.

Descripción: Al presionar el botón de Inicio, el emulador mostró la pantalla principal del celular y en Logcat aparecieron onPause (la actividad pierde el foco) y onStop (ya no se ve). Lo importante es lo que no apareció: onDestroy. La aplicación no se cerró, solo pasó a segundo plano y sigue guardada en la memoria, igual que una tienda con la cortina abajo.

Prueba 3: regresar a la aplicación (botón Recientes □)

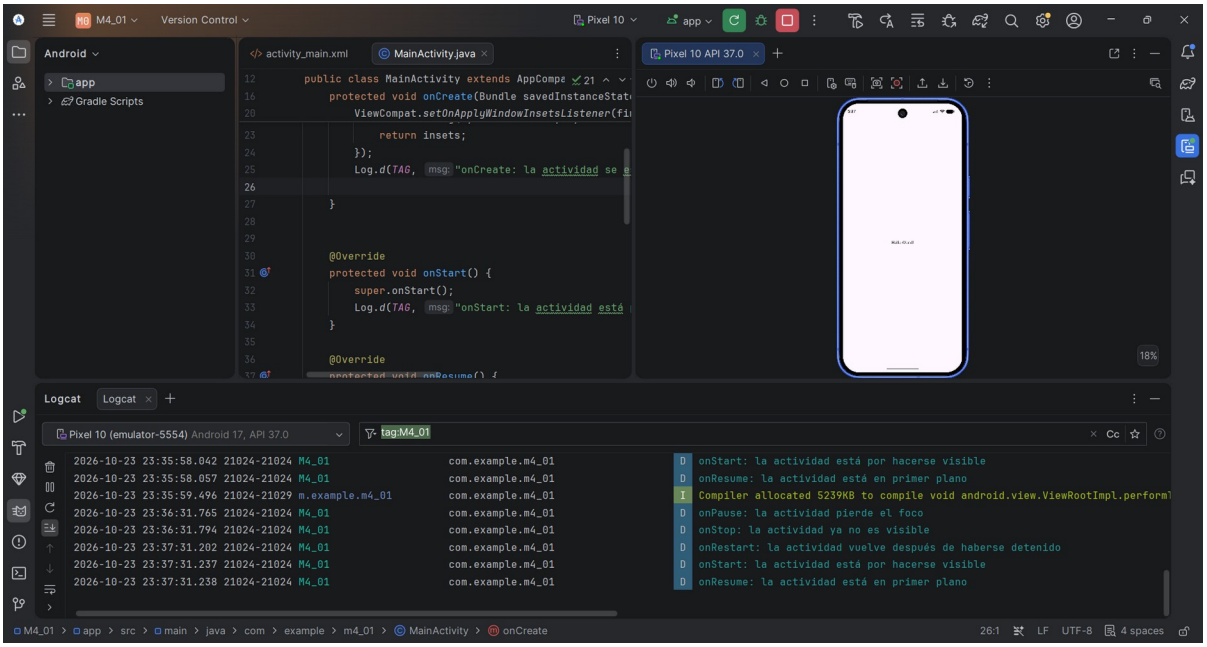


Figura 9. Al regresar a la aplicación: onRestart → onStart → onResume.

Descripción: Desde la lista de aplicaciones recientes se volvió a abrir la aplicación. Esta vez no apareció onCreate, sino onRestart, porque la actividad ya existía y no hacía falta crearla de nuevo. Después se repitieron onStart y onResume, ya que la actividad volvió a hacerse visible y a quedar en primer plano.

Prueba 4: girar la pantalla (botón Rotar)

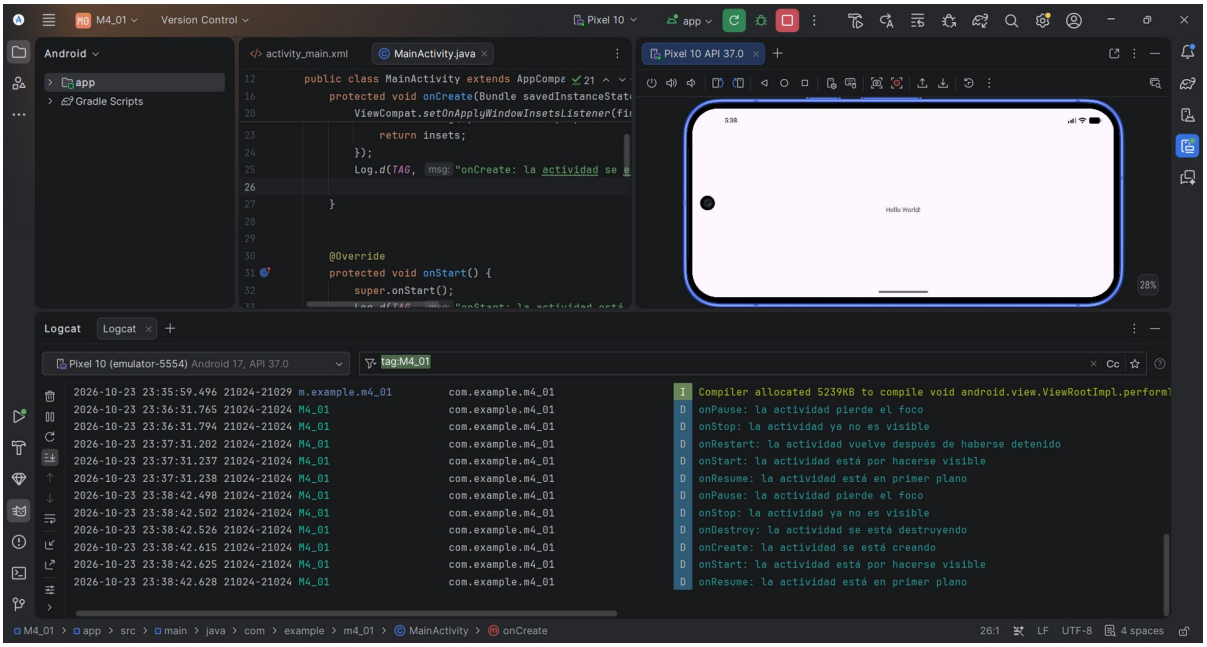


Figura 10. Al girar la pantalla: onPause → onStop → onDestroy → onCreate → onStart → onResume.

Descripción: Al girar el emulador, la actividad recorrió el ciclo completo: se pausó, se detuvo, se destruyó (onDestroy) y, enseguida, se volvió a crear desde cero (onCreate, onStart y onResume). Con esta prueba se completaron los siete métodos del ciclo de vida.

4.5. Lo que se descubrió con las pruebas

Más allá de ver los mensajes, las pruebas dejaron varias observaciones que no son evidentes a simple vista:

- **El orden nunca cambia.** Siempre se llama onStart antes de onResume, y onPause antes de onStop. Android sigue el mismo camino del diagrama en todas las situaciones.
- **Minimizar no es cerrar.** Al presionar Inicio, la aplicación solo se detiene (onStop) y se queda en memoria. Por eso, cuando volvemos a una aplicación en el celular, normalmente está justo donde la dejamos.
- **onRestart solo aparece al regresar.** No se ejecuta al abrir la aplicación por primera vez, únicamente cuando vuelve después de un onStop. Es la forma en que Android distingue "abrir por primera vez" de "regresar".
- **Girar la pantalla destruye la actividad.** Este fue el descubrimiento más sorprendente. Android destruye la pantalla y la crea de nuevo para acomodar el diseño a la nueva orientación. Esto explica por qué, en algunas aplicaciones mal programadas, se pierde lo que se había escrito en un formulario al girar el celular: todo se vuelve a crear desde cero. Los programadores deben guardar esa información antes de que la actividad se destruya.
- **El botón Atrás no siempre destruye la actividad.** En las versiones recientes de Android, salir de la pantalla principal con el botón Atrás generalmente solo manda la aplicación a segundo plano, sin llamar a onDestroy. Por eso, para observar onDestroy se utilizó la rotación de pantalla.

Código fuente de esta actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M4-01

5. Conclusión

Esta fue de las actividades más confusas, y por lo mismo fue en la que cometí más errores. Al principio no entendía qué hacía o qué debía hacer la aplicación, y lo peor fue no encontrar el error que había cometido: eran las llaves, que dejaron los métodos fuera de la clase, hasta que con Ctrl + Alt + L pude acomodar el código y verlo.

Me di cuenta de que hacer una aplicación para celular es más complejo de lo que parece en teoría. Nunca había desarrollado un programa ejecutable (.exe) para computadora, y menos una aplicación para celular, y no imaginaba que también había que pensar en lo que pasa cuando el usuario la minimiza, regresa a ella o gira el teléfono, porque Android ejecuta métodos distintos en cada caso sin que el programador los llame.

Lo que más se me hizo extraño fue que, al girar la pantalla, la actividad se destruye y se vuelve a crear en ese mismo instante. En Logcat aparecieron onPause, onStop y onDestroy, y enseguida otra vez onCreate, onStart y onResume. Eso explica por qué en algunas aplicaciones se pierde lo que se había escrito al girar el celular.

6. Referencias

Caules, C. Á. (2025, 12 de noviembre). *Java constructores this() y super()*. Arquitectura Java. https://www.arquitecturajava.com/java-constructores-y-super/

Charly Cimino. (2022, 2 de febrero). *SUPER en Java. Los constructores NO se heredan* [Video]. YouTube. https://www.youtube.com/watch?v=fUUibK4xPq8

Ciclo de vida de la actividad. (s. f.). Android Developers. https://developer.android.com/guide/components/activities/activity-lifecycle?hl=es-419

Cristian Henao. (2022, 20 de abril). *#12. Qué son las Activities y cuál es su ciclo de vida* [Video]. YouTube. https://www.youtube.com/watch?v=38y6LFSXT1g

Google. (s. f.-a). *Handle configuration changes*. Android Developers. https://developer.android.com/guide/topics/resources/runtime-changes

Google. (s. f.-b). *View logs with Logcat*. Android Developers. https://developer.android.com/studio/debug/logcat

La Geekipedia De Ernesto. (2017, 17 de noviembre). *Curso Android desde cero #6 | Ciclo de vida de un Activity* [Video]. YouTube. https://www.youtube.com/watch?v=poipVVd2jzU

Programador Novato. (2021, 9 de septiembre). *Eventos que se ejecutan al iniciar nuestra aplicación en Android Studio [04]* [Video]. YouTube. https://www.youtube.com/watch?v=uv0DKmqPthE